

Dokumentacja: Git

Spis treści

- 1. Wprowadzenie do Gita
 - 1.1. Czym jest Git i do czego służy
 - 1.2. Instalacja Gita
 - 1.3. Konfiguracja wstępna — `git config`
 - 1.4. Kluczowe pojęcia: repozytorium, commit, branch, HEAD
 - 1.5. Obszary pracy w Gicie — Working Directory, Staging Area, Repository
- 2. Pierwsze repozytorium
 - 2.1. Tworzenie repozytorium — `git init`
 - 2.2. Klonowanie istniejącego repozytorium — `git clone`
 - 2.3. Sprawdzanie stanu — `git status`
 - 2.4. Plik `.gitignore` — co pomijamy
- 3. Zapisywanie zmian — commit
 - 3.1. Dodawanie plików do staging — `git add`
 - 3.2. Tworzenie commitu — `git commit`
 - 3.3. Przeglądanie historii — `git log`
 - 3.4. Porównywanie zmian — `git diff`
 - 3.5. Cofanie zmian — `git restore`, `git reset`, `git revert`
- 4. Gałęzie — branch
 - 4.1. Czym jest gałąź i po co jej używać
 - 4.2. Tworzenie i przełączanie gałęzi — `git branch`, `git checkout`, `git switch`
 - 4.3. Scalanie gałęzi — `git merge`
 - 4.4. Konflikty przy scalaniu i jak je rozwiązywać
 - 4.5. Usuwanie gałęzi
- 5. Praca ze zdalnym repozytorium
 - 5.1. Dodawanie zdalnego repozytorium — `git remote`
 - 5.2. Wysyłanie zmian — `git push`
 - 5.3. Pobieranie zmian — `git fetch` i `git pull`
 - 5.4. GitHub — tworzenie repozytorium i pierwsze push
- 6. Przydatne komendy i wzorce
 - 6.1. Zmiana ostatniego commitu — `git commit --amend`
 - 6.2. Schowek tymczasowy — `git stash`
 - 6.3. Tagi — `git tag`
 - 6.4. Aliasy — skróty do komend
 - 6.5. Typowy przepływ pracy — ściągawka krok po kroku

1. Wprowadzenie do Gita

1.1. Czym jest Git i do czego służy

Git to rozproszony system kontroli wersji (ang. *Distributed Version Control System*). Pozwala śledzić zmiany w plikach projektu w czasie, cofać się do poprzednich wersji oraz współpracować z innymi programistami bez nadpisywania nawzajem swojej pracy.

Co Git robi dla Ciebie:

- Zapisuje historię każdej zmiany (kto, co, kiedy zmienił)
- Pozwala eksperymentować na osobnych gałęziach bez ryzyka zepsucia głównego kodu

- Umożliwia pracę równoległą — kilka osób jednocześnie modyfikuje projekt
- Pozwala wrócić do dowolnego wcześniejszego stanu projektu

Git ≠ GitHub — Git to narzędzie działające lokalnie na Twoim komputerze. GitHub, GitLab, Bitbucket to serwisy internetowe, które *przechowują* repozytoria Git w chmurze i dodają funkcje społecznościowe (pull requesty, issues itp.).

1.2. Instalacja Gita

Windows:

1. Pobierz instalator ze strony git-scm.com
2. Uruchom instalator — domyślne ustawienia są odpowiednie
3. Po instalacji otwórz **Git Bash** lub zwykły terminal CMD/PowerShell

macOS:

```
# Przez Homebrew (jeśli zainstalowany):  
brew install git  
  
# Lub zainstaluj Xcode Command Line Tools:  
xcode-select --install
```

Linux (Debian/Ubuntu):

```
sudo apt update  
sudo apt install git
```

Sprawdzenie wersji po instalacji:

```
git --version  
# Przykładowy wynik: git version 2.43.0
```

1.3. Konfiguracja wstępna — `git config`

Przed pierwszym użyciem należy podać swoje dane — Git dołącza je do każdego commitu.

```
# Ustaw imię i nazwisko (widoczne w historii commitów)  
git config --global user.name "Jan Kowalski"  
  
# Ustaw adres e-mail  
git config --global user.email "jan.kowalski@example.com"  
  
# Ustaw domyślny edytor tekstu (np. VS Code)  
git config --global core.editor "code --wait"  
  
# Ustaw domyślną nazwę głównej gałęzi na "main"  
git config --global init.defaultBranch main
```

Flaga `--global` oznacza, że ustawienie dotyczy wszystkich repozytoriów na tym komputerze. Bez flagi — ustawienie dotyczy tylko bieżącego repozytorium.

Sprawdzenie konfiguracji:

```
git config --list
# Wyświetla wszystkie ustawienia

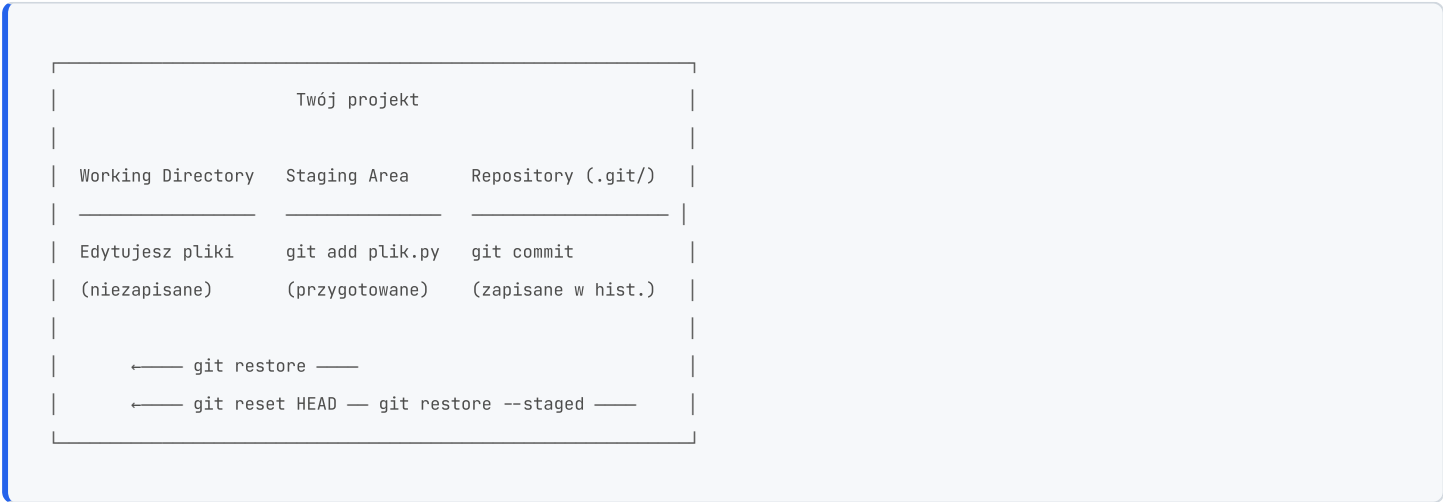
git config user.name
# Wyświetla tylko imię
```

1.4. Kluczowe pojęcia: repozytorium, commit, branch, HEAD

POJĘCIE	WYJAŚNIENIE
Repozytorium (repo)	Folder projektu śledzony przez Gita — zawiera wszystkie pliki i całą historię zmian
Commit	Migawka (snapshot) stanu projektu w danym momencie — ma unikalny hash (np. <code>a3f5c2d</code>), autora i wiadomość
Branch (gałąź)	Oddzielna linia rozwoju projektu — pozwala pracować nad nową funkcją bez wpływu na główny kod
HEAD	Wskaźnik na bieżący commit (czyli „gdzie teraz jesteś” w historii) — zwykle wskazuje na wierzchołek aktywnej gałęzi
Staging Area	Poczekalnia — pliki dodane przez <code>git add</code> , które wejdą do następnego commitu
Remote	Zdalne repozytorium (np. na GitHubie) — źródło/cel synchronizacji
Hash / SHA	Unikalny 40-znakowy identyfikator każdego commitu (używa się skróconego 7-znakowego, np. <code>a3f5c2d</code>)
Tag	Etykieta przypisana do konkretnego commitu (zwykle wersja: <code>v1.0</code> , <code>v2.3.1</code>)

1.5. Obszary pracy w Gicie — Working Directory, Staging Area, Repository

Git różni trzy obszary, przez które przechodzą Twoje zmiany:



- 1. Working Directory** — tu edytujesz pliki. Zmiany są widoczne, ale Git ich jeszcze nie śledzi jako „gotowe do zapisu”.
- 2. Staging Area** (indeks) — pliki dodane przez `git add`. To „poczekalnia” przed commitem. Możesz tu selektywnie wybierać, które zmiany wejdą do commitu.
- 3. Repository** — historia commitów zapisana w ukrytym folderze `.git/`. Stały, nienaruszalny zapis.

2. Pierwsze repozytorium

2.1. Tworzenie repozytorium — `git init`

```
# 1. Przejdź do folderu projektu
cd /ścieżka/do/mojego/projektu

# 2. Zainicjalizuj repozytorium
git init
# Wynik: Initialized empty Git repository in /ścieżka/.git/

# Lub stwórz nowy folder i od razu zainicjalizuj:
git init nazwa-projektu
cd nazwa-projektu
```

Po wykonaniu `git init` powstaje ukryty folder `.git/` — to serce repozytorium. **Nie usuwaj go ręcznie.**

Przykład — pierwsze kroki od zera:

```
mkdir moj-projekt
cd moj-projekt
git init

# Stwórz pierwszy plik
echo "# Mój projekt" > README.md

# Dodaj do staging
git add README.md

# Zapisz jako pierwszy commit
git commit -m "Pierwszy commit: dodanie README"
```

2.2. Klonowanie istniejącego repozytorium — `git clone`

```
# Klonuj repozytorium z GitHuba (HTTP)
git clone https://github.com/uzytkownik/nazwa-repo.git

# Klonuj do konkretnego folderu
git clone https://github.com/uzytkownik/nazwa-repo.git moj-folder

# Klonuj przez SSH (wymaga skonfigurowanego klucza SSH)
git clone git@github.com:uzytkownik/nazwa-repo.git
```

Klonowanie pobiera cały projekt wraz z **pełną historią** commitów. Po klonowaniu masz gotową lokalną kopię — od razu możesz pracować.

2.3. Sprawdzanie stanu — `git status`

`git status` to najczęściej używana komenda — pokazuje co dzieje się w repozytorium.

```
git status
```

Możliwe stany pliku:

STAN	ZNACZENIE
Untracked	Nowy plik, Git go jeszcze nie zna
Modified	Plik śledzony, ale zmieniony od ostatniego commitu
Staged	Dodany do staging (przez <code>git add</code>), gotowy do commitu
Unmodified	Brak zmian od ostatniego commitu

Przykładowe wyjście:

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   plik.py

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
    modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    notes.txt
```

2.4. Plik `.gitignore` — co pomijamy

Plik `.gitignore` określa, które pliki i foldery Git ma **ignorować** (nie śledzić).

```
# Tworzenie .gitignore w katalogu projektu
touch .gitignore
```

Przykładowa zawartość `.gitignore` dla projektu Python:

```
# Środowisko wirtualne
venv/
.env/
env/

# Pliki skompilowane
__pycache__/
*.pyc
*.pyo
*.pyd

# Pliki systemowe
```

```
.DS_Store
Thumbs.db

# Pliki konfiguracyjne z hasłami/kluczami
.env
secrets.json
config.local.py

# Pliki IDE
.idea/
.vscode/
*.suo
*.user
```

Przykładowa zawartość `.gitignore` dla projektu C# / WPF:

```
# Visual Studio
bin/
obj/
*.user
*.suo
.vs/

# NuGet
packages/
*.nupkg

# Pliki lokalne
*.local.config
appsettings.local.json
```

Reguły składni `.gitignore`:

WZORZEC	CO IGNORUJE
<code>*.log</code>	Wszystkie pliki <code>.log</code>
<code>build/</code>	Folder o nazwie <code>build</code>
<code>**/temp</code>	Folder <code>temp</code> w dowolnym miejscu
<code>!ważny.log</code>	Wyjątek — ten plik NIE jest ignorowany
<code>doc/*.txt</code>	Pliki <code>.txt</code> tylko w folderze <code>doc/</code>

Ważne: Jeśli plik był już śledzony przez Gita, dodanie go do `.gitignore` nie wystarczy — trzeba go najpierw usunąć z indeksu: `git rm --cached plik.txt`

3. Zapisywanie zmian — commit

3.1. Dodawanie plików do staging — `git add`

```
# Dodaj konkretny plik
git add main.py

# Dodaj kilka plików
git add main.py utils.py README.md

# Dodaj cały folder
git add src/

# Dodaj WSZYSTKIE zmienione i nowe pliki
git add .

# Dodaj wszystkie pliki z rozszerzeniem .py
git add *.py

# Tryb interaktywny – wybierasz które części pliku dodać
git add -p main.py
```

Usuwanie z staging (bez utraty zmian w pliku):

```
git restore --staged main.py

# lub (starsza składnia):
git reset HEAD main.py
```

3.2. Tworzenie commitu — `git commit`

```
# Commit z wiadomością (zalecane)
git commit -m "Dodanie funkcji logowania użytkownika"

# Commit z dłuższym opisem (otwiera edytor)
git commit

# Skrót: add wszystkich zmienionych (śledzonych) plików + commit
git commit -am "Poprawka błędu w obliczeniach"

# UWAGA: -a nie dodaje nowych (Untracked) plików!
```

Zasady dobrego komunikatu commitu:

- Zaczynij od czasownika w trybie rozkazującym: *Dodaj, Napraw, Usuń, Zmień, Refaktoru*
- Pierwsza linia: maks. 72 znaki
- Opisuj **co i dlaczego**, nie jak

```
# ✅ Dobre wiadomości:
git commit -m "Dodaj walidację formularza logowania"
git commit -m "Napraw błąd dzielenia przez zero w kalkulatorze"
```

```
git commit -m "Usuń zbędne importy z main.py"
```

```
# ❌ Złe wiadomości:
```

```
git commit -m "fix"
```

```
git commit -m "zmiany"
```

```
git commit -m "aaa"
```

3.3. Przeglądanie historii — `git log`

```
# Pełna historia commitów
```

```
git log
```

```
# Skrócona, jedna linia na commit
```

```
git log --oneline
```

```
# Z grafem gałęzi
```

```
git log --oneline --graph --all
```

```
# Ostatnie N commitów
```

```
git log -5
```

```
# Historia konkretnego pliku
```

```
git log --oneline main.py
```

```
# Commity od konkretnej daty
```

```
git log --since="2024-01-01" --until="2024-12-31"
```

```
# Commity konkretnego autora
```

```
git log --author="Kowalski"
```

Przykładowe wyjście `git log --oneline`:

```
a3f5c2d Napraw błąd w module kalkulatora
```

```
8b2e1a0 Dodaj obsługę wyjątków w parsowaniu pliku
```

```
f7c3d5b Refaktoryzacja klasy Album
```

```
3a1b9c2 Pierwszy commit: dodanie README
```

Pokazanie szczegółów konkretnego commitu:

```
git show a3f5c2d
```

```
# lub ostatni commit:
```

```
git show HEAD
```


3.4. Porównywanie zmian — `git diff`

```
# Zmiany w Working Directory (niezastagedowane)
git diff

# Zmiany w Staging Area (zastagedowane, gotowe do commitu)
git diff --staged

# lub:
git diff --cached

# Porównanie dwóch commitów
git diff a3f5c2d 8b2e1a0

# Porównanie gałęzi
git diff main feature/logowanie

# Zmiany w konkretnym pliku
git diff main.py
```

Jak czytać wynik `git diff`:

```
diff --git a/main.py b/main.py
index 3a1b9c2..f7c3d5b 100644
--- a/main.py      +- stara wersja
+++ b/main.py      +- nowa wersja
@@ -10,7 +10,8 @@
def oblicz(a, b):
-     return a + b      +- usunięta linia (czerwona)
+     if b == 0:        +- dodana linia (zielona)
+         return None
+     return a / b
```

3.5. Cofanie zmian — `git restore`, `git reset`, `git revert`

Odrzucenie zmian w Working Directory (nie zapisane, przed `git add`):

```
# Przywróć plik do stanu z ostatniego commitu
git restore main.py

# Przywróć wszystkie pliki
git restore .
```

⚠ `git restore` jest **nieodwracalne** — niezapisane zmiany przepadają!

Usunięcie pliku z Staging Area (po `git add`, przed `git commit`):

```
# Usuń z staging, ale zachowaj zmiany w pliku
```

```
git restore --staged main.py
```

Cofnięcie commitów — `git reset`:

```
# Cofnij ostatni commit, zachowaj zmiany w staging
git reset --soft HEAD~1

# Cofnij ostatni commit, zachowaj zmiany w Working Directory
git reset --mixed HEAD~1
# (to jest domyślne zachowanie bez flagi)

# Cofnij ostatni commit i USUŃ zmiany całkowicie
git reset --hard HEAD~1
```

FLAGA	STAGING AREA	WORKING DIRECTORY
<code>--soft</code>	Zachowane (pliki staged)	Bez zmian
<code>--mixed</code>	Wyczyszczone (pliki unstaged)	Bez zmian
<code>--hard</code>	Wyczyszczone	Zmiany usunięte!

Bezpieczne cofnięcie — `git revert` (zalecane na współdzielonych gałęziach):

```
# Tworzy NOWY commit, który odwraca zmiany z podanego commitu
git revert a3f5c2d

# Cofnij ostatni commit (tworząc nowy "odwrotny" commit)
git revert HEAD
```

`git revert` jest **bezpieczny** — nie niszczy historii, tylko dodaje nowy commit. `git reset --hard` przepisuje historię — **nie używaj** na commitach już wystanych na zdalne repo!

4. Gałęzie — branch

4.1. Czym jest gałąź i po co jej używać

Gałąź (branch) to oddzielna „linia czasu” projektu. Główna gałąź to zazwyczaj `main` (lub `master` w starszych projektach).

```
main:    A — B — C ————— G (merge)
          |
          |   D — E   |
feature/logowanie:      (nowa funkcja)
```

Kiedy tworzyć nową gałąź:

- Chcesz dodać nową funkcję bez ryzyka zepsucia działającego kodu
- Naprawiasz błąd (hotfix)
- Eksperymentujesz z nowym podejściem
- Pracujesz w zespole — każda osoba ma swoją gałąź

4.2. Tworzenie i przełączanie gałęzi — `git branch`, `git checkout`, `git switch`

```
# Wyświetl wszystkie lokalne gałęzie (* oznacza aktywną)
git branch

# Wyświetl lokalne i zdalne gałęzie
git branch -a

# Utwórz nową gałąź (bez przełączania)
git branch feature/logowanie

# Przełącz się na gałąź
git switch feature/logowanie
# lub starsza składnia:
git checkout feature/logowanie

# Utwórz nową gałąź I od razu się przełącz (najczęściej używane)
git switch -c feature/logowanie
# lub starsza składnia:
git checkout -b feature/logowanie

# Wróć do poprzedniej gałęzi
git switch -
```

Typowy przepływ z gałęziami:

```
# 1. Jesteś na main — upewnij się że jest aktualny
git switch main
git pull

# 2. Utwórz nową gałąź dla nowej funkcji
git switch -c feature/koszyk-zakupow

# 3. Pracuj, dodawaj commity...
git add .
git commit -m "Dodaj klasę Koszyk"
git add .
git commit -m "Dodaj metodę dodajProdukt"

# 4. Wróć do main i scal
git switch main
git merge feature/koszyk-zakupow

# 5. Usuń gałąź po scaleniu
git branch -d feature/koszyk-zakupow
```

4.3. Scalanie gałęzi — `git merge`

```
# Będąc na gałęzi docelowej (np. main), scal z feature
git switch main
git merge feature/logowanie
```

Rodzaje scalania:

Fast-forward (brak nowego commitu — historia jest liniowa):

```
Przed:  main: A — B
          └─ C — D (feature)
```

```
Po:      main: A — B — C — D
```

Merge commit (gdy obie gałęzie miały nowe commity):

```
Przed:  main: A — B — E
          └─ C — D (feature)
```

```
Po:      main: A — B — E — M (merge commit)
          └─ C — D ─┘
```

```
# Wymuś zawsze merge commit (zachowanie historii gałęzi)
git merge --no-ff feature/logowanie

# Podejrzyj co zostanie scalone (bez faktycznego scalania)
git merge --dry-run feature/logowanie
```

4.4. Konflikty przy scalaniu i jak je rozwiązywać

Konflikt wystąpi, gdy ta sama linia w pliku została zmieniona w obu gałęziach.

```
git merge feature/logowanie

# Auto-merging main.py
# CONFLICT (content): Merge conflict in main.py
# Automatic merge failed; fix conflicts and then commit the result.
```

Jak wygląda konflikt w pliku:

```
def oblicz_rabat(cena):
<<<<<<< HEAD
    # Wersja z gałęzi main
    return cena * 0.9
=====
    # Wersja z gałęzi feature/logowanie
```

```
return cena * 0.85

>>>>> feature/logowanie
```

Rozwiązanie konfliktu krok po kroku:

1. Otwórz plik z konfliktem w edytorze
2. Zdecyduj która wersja jest prawidłowa (lub napisz nową)
3. Usuń znaczniki <<<<<<, =====, >>>>>>
4. Zapisz plik
5. `git add plik_z_konfliktem.py`
6. `git commit -m "Rozwiązanie konfliktu w oblicz_rabat"`

```
# Sprawdź które pliki mają konflikty
git status

# Przerwij scalanie (wróć do stanu sprzed merge)
git merge --abort
```

4.5. Usuwanie gałęzi

```
# Usuń lokalną gałąź (tylko jeśli jest już scalona)
git branch -d feature/logowanie

# Wymuś usunięcie (nawet jeśli nieskalona)
git branch -D feature/logowanie

# Usuń zdalną gałąź
git push origin --delete feature/logowanie
```

5. Praca ze zdalnym repozytorium

5.1. Dodawanie zdalnego repozytorium — `git remote`

```
# Wyświetl zdalne repozytoria
git remote -v

# Dodaj zdalne repozytorium (zwykle origin)
git remote add origin https://github.com/uzytkownik/projekt.git

# Zmień URL zdalnego repozytorium
git remote set-url origin https://github.com/uzytkownik/novy-url.git

# Usuń zdalne repozytorium
git remote remove origin
```

```
# Zmień nazwę
git remote rename origin backup
```

Nazwa `origin` to konwencja — technicznie możesz użyć dowolnej nazwy.

5.2. Wysyłanie zmian — `git push`

```
# Wyślij lokalną gałąź na zdalne repo
git push origin main

# Wyślij i ustaw upstream (śledzenie) — potem wystarczy "git push"
git push -u origin main

# Po ustawieniu upstream — skrót
git push

# Wyślij nową lokalną gałąź na zdalne repo
git push -u origin feature/logowanie

# Wymuś push (UWAGA: nadpisuje historię na serwerze!)
git push --force

# Bezpieczniejsza wersja (nie nadpisze jeśli ktoś już coś wypchnął):
git push --force-with-lease
```

⚠ `git push --force` nigdy nie używaj na gałęziach współdzielonych z innymi (np. `main`)!

5.3. Pobieranie zmian — `git fetch` i `git pull`

```
# git fetch — pobiera zmiany ze zdalnego repo BEZ scalania
git fetch origin

# Po fetch możesz zobaczyć co się zmieniło:
git log HEAD..origin/main --oneline

# Scal pobrane zmiany ręcznie:
git merge origin/main
```

```
# git pull — pobiera i od razu scala (fetch + merge)
git pull

# git pull z konkretnej gałęzi
git pull origin main

# Pobierz i rebase zamiast merge (liniowa historia)
git pull --rebase
```

Kiedy używać fetch vs pull:

- `git fetch` — gdy chcesz **najpierw zobaczyć** co się zmieniło, zanim scalisz
- `git pull` — gdy chcesz szybko zsynchronizować i ufasz że nie będzie konfliktów

5.4. GitHub — tworzenie repozytorium i pierwsze push

Scenariusz 1: Nowy projekt — najpierw lokalnie:

```
# 1. Utwórz lokalnie
mkdir moj-projekt && cd moj-projekt
git init
echo "# Mój Projekt" > README.md
git add .
git commit -m "Pierwszy commit"

# 2. Na GitHubie: kliknij "New repository", skopiuj URL

# 3. Połącz i wyślij
git remote add origin https://github.com/TY/moj-projekt.git
git branch -M main
git push -u origin main
```

Scenariusz 2: Istniejące repo na GitHubie — sklonuj:

```
git clone https://github.com/TY/moj-projekt.git
cd moj-projekt
# Pracuj, commituj, push...
git push
```

Standardowy codzienny cykl pracy z GitHubem:

```
# 1. Zaczynij dzień – pobierz zmiany
git pull

# 2. Pracuj i commituj
git add .
git commit -m "Opis zmiany"

# 3. Wyślij na koniec dnia
git push
```

6. Przydatne komendy i wzorce

6.1. Zmiana ostatniego commitu — `git commit --amend`

```
# Zmień wiadomość ostatniego commitu
git commit --amend -m "Poprawiona wiadomość"
```

```
# Dodaj zapomniane pliki do ostatniego commitu (bez zmiany wiadomości)
git add zapomniany_plik.py
git commit --amend --no-edit
```

⚠ Używaj `--amend` tylko jeśli commit **nie został jeszcze wypchnięty** na zdalne repo!

6.2. Schowek tymczasowy — `git stash`

`git stash` pozwala tymczasowo odłożyć niezakończone zmiany i wrócić do czystego Working Directory.

```
# Odłóż zmiany do schowka
git stash

# Odłóż z opisem
git stash push -m "Prace nad formularzem logowania"

# Wyświetl zawartość schowka
git stash list

# Wynik:
# stash@{0}: On main: Prace nad formularzem logowania
# stash@{1}: WIP on main: a3f5c2d Poprzedni commit

# Przywróć ostatnio odłożone zmiany
git stash pop

# Przywróć konkretny schowek
git stash pop stash@{1}

# Przywróć bez usuwania ze schowka
git stash apply

# Usuń konkretny schowek
git stash drop stash@{0}

# Usuń wszystkie schowki
git stash clear
```

Typowy scenariusz użycia `git stash`:

```
# Pracujesz nad nową funkcją, nagle musisz pilnie naprawić błąd na main

git stash                # odłóż bieżące zmiany
git switch main          # przejdź na main
git switch -c hotfix/blad # utwórz gałąź hotfix
# ... napraw błąd ...
git commit -am "Napraw krytyczny błąd w module płatności"
git switch main
git merge hotfix/blad
```



```
git switch feature/moja-funkcja # wróć do swojej pracy
git stash pop                  # przywróć odłożone zmiany
```

6.3. Tagi — `git tag`

Tagi służą do oznaczania ważnych punktów w historii, zwykle wersji oprogramowania.

```
# Wyświetl wszystkie tagi
git tag

# Utwórz lekki tag (tylko etykieta)
git tag v1.0

# Utwórz tag z opisem (zalecane)
git tag -a v1.0 -m "Wersja 1.0 – pierwsze publiczne wydanie"

# Oznacz starszy commit tagiem
git tag -a v0.9 a3f5c2d -m "Wersja 0.9 beta"

# Wyślij tagi na zdalne repo (git push ich domyślnie nie wysyła!)
git push origin v1.0
git push origin --tags # wyślij wszystkie tagi

# Usuń tag lokalnie
git tag -d v1.0

# Usuń tag z zdalnego repo
git push origin --delete v1.0

# Przejdź do stanu projektu z danego tagu
git checkout v1.0
```

6.4. Aliasy — skróty do komend

```
# Utwórz skróty dla często używanych komend
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.cm "commit -m"
git config --global alias.lg "log --oneline --graph --all"
git config --global alias.last "log -1 HEAD"
git config --global alias.unstage "restore --staged"
```

Po konfiguracji możesz pisać:

```
git st      # zamiast: git status
git co main  # zamiast: git checkout main
git lg      # zamiast: git log --oneline --graph --all
```

```
git last          # pokazuje ostatni commit
git unstage plik.py # zamiast: git restore --staged plik.py
```

6.5. Typowy przepływ pracy — ściągawka krok po kroku

Inicjalizacja projektu od zera:

```
mkdir projekt && cd projekt
git init
echo "# Projekt" > README.md
echo "__pycache__/" > .gitignore
git add .
git commit -m "Inicjalizacja projektu"
git remote add origin https://github.com/TY/projekt.git
git push -u origin main
```

Codzienna praca:

```
git pull          # pobierz zmiany od innych
git switch -c feature/nowa-f # utwórz gałąź dla nowej funkcji
# ... edytuj pliki ...
git status        # sprawdź co zmieniłeś
git diff          # podejrzuj zmiany
git add .         # dodaj do staging
git commit -m "Opis zmiany" # zapisz commit
git push -u origin feature/nowa-f # wyślij gałąź na GitHub
```

Scalanie gotowej funkcji:

```
git switch main    # przejdź na main
git pull          # upewnij się że main jest aktualny
git merge feature/nowa-f # scal funkcję
git push          # wyślij zaktualizowany main
git branch -d feature/nowa-f # usuń lokalną gałąź
git push origin --delete feature/nowa-f # usuń zdalną gałąź
```

Szybka ściągawka komend

KOMENDA	OPIS
<code>git init</code>	Inicjalizuj nowe repozytorium
<code>git clone <url></code>	Sklonuj zdalne repozytorium
<code>git status</code>	Sprawdź stan plików
<code>git add .</code>	Dodaj wszystkie zmiany do staging
<code>git add <plik></code>	Dodaj konkretny plik do staging
<code>git commit -m "opis"</code>	Zapisz commit z wiadomością
<code>git log --oneline</code>	Historia commitów (krótko)
<code>git diff</code>	Pokaż niezastagedowane zmiany
<code>git diff --staged</code>	Pokaż zastagedowane zmiany
<code>git branch</code>	Wyświetl gałęzie
<code>git switch -c <nazwa></code>	Utwórz i przełącz na nową gałąź
<code>git switch <nazwa></code>	Przełącz na gałąź
<code>git merge <gałąź></code>	Scal gałąź z bieżącą
<code>git push</code>	Wyślij commity na zdalne repo
<code>git pull</code>	Pobierz i scal ze zdalnego repo
<code>git fetch</code>	Pobierz zmiany (bez scalania)
<code>git restore <plik></code>	Cofnij zmiany w pliku (Working Dir)
<code>git restore --staged <plik></code>	Usuń plik z staging
<code>git reset --soft HEAD~1</code>	Cofnij ostatni commit (zachowaj staged)
<code>git reset --hard HEAD~1</code>	Cofnij ostatni commit (usuń zmiany)
<code>git revert HEAD</code>	Cofnij commit tworząc nowy commit
<code>git stash</code>	Odlóż zmiany do schowka
<code>git stash pop</code>	Przywróć ze schowka
<code>git tag -a v1.0 -m "..."</code>	Utwórz opisany tag
<code>git remote -v</code>	Wyświetl zdalne repozytoria
<code>git remote add origin <url></code>	Dodaj zdalne repo